YOLO Web Builder

A web-based builder for generating runnable **Ultralytics YOLOv8 / YOLO11 detection** training bundles.

This project provides a browser UI for configuring Roboflow datasets, selecting per-target training/export options, saving project snapshots, generating a downloadable bundle, and optionally uploading that bundle to a remote Linux machine over SSH for execution.

Docker Deploy Guide

What this project does

- Builds **Detection-only** YOLO pipelines.
- Supports **YOLOv8** and **YOLO11** model families.
- Lets you define one or more **single-class runs** by `Class name` (`role`).
- Can additionally build a **Hybrid** run that merges all selected class groups into one multi-class dataset.
- Generates runnable scripts for:
  - dataset download
  - dataset merge / balancing
  - training
  - ONNX export
  - TensorRT engine export
  - TFLite export
  - artifact collection and packaging
- Can run the generated bundle through **SSH** and expose:
  - remote status
  - remote log download
  - remote artifacts download
- Saves project configurations into `backend/jobs/projects/`.

Observed architecture

From the current repository structure, the project is organized into two main layers:

1. **Web application layer**
   - `backend/app.py`: FastAPI server
   - `backend/templates/index.html`: UI shell
   - `backend/static/app.js`: multi-step wizard logic
   - `backend/core/`: request models, bundle generation, SSH execution, security helpers
2. **Pipeline template layer**
   - `pipeline_templates/`: files copied into generated bundles
   - Includes the training/export entry scripts, merge/balance tools, artifact collection, and packaging scripts

The Docker path installs only `backend/requirements.txt`, so the web server runtime is intentionally lightweight. The generated training bundle uses its own `requirements_pipeline.txt`.

Prerequisites (At the target machine/environment)

1. **Git pull the Project structure.**

- The first time Git.

```
cd /<YOUR DIRECTORY NAME>
git clone git@github.com:tku411770448/Training-models-with-roboflow-via-remote-control-.git
```

- Later

```
cd /<YOUR DIRECTORY NAME>
git pull
```

2. **Create the exact Python environment that will be used by SSH execution, then install the dependencies into that same interpreter.**

requirements.txt

```
conda create -n <YOUR PYTHON ENVIRONMENT NAME> python=3.10
conda activate <YOUR PYTHON ENVIRONMENT NAME>
python -m pip install -r requirements.txt
```

Important compatibility note

When you run the bundle through SSH, the backend will use the exact Python executable you provide in the UI. For example, if you set:

```
/home/tkupzj0609/miniconda3/envs/yolov11/bin/python
```

then the required packages must be installed for **that exact interpreter and environment**. In other words, the package set must be compatible with the SSH runtime environment itself; otherwise the remote run will fail during import or pipeline execution.

For that reason, always install packages with the target interpreter explicitly, for example:

```
/home/tkupzj0609/miniconda3/envs/yolov11/bin/python -m pip install -r requirements.txt
```

Do **not** rely on packages being installed under `~/.local` (for example `~/.local/lib/python3.10/site-packages`). Python user-site installs are typically placed under `~/.local/...`, and `pip install --user` targets that location. However, when a remote job is expected to run inside a specific conda or virtual environment interpreter, the required packages should be installed into that interpreter's own `site-packages`, not into a separate `.local` user site. If the required packages are installed only in `.local` instead of the designated environment path, the program may still fail even though the packages appear to be present.

Main features

1. Project-based wizard UI

The front end is a step-based builder with save/load behavior and bilingual UI text (Chinese / English).

2. Roboflow dataset configuration

Each dataset card supports:

- API key
- workspace
- project
- version
- format (`YOLOv8` / `YOLO11` in UI)
- class grouping via `role`

The UI also includes Roboflow URL / snippet parsing helpers.

3. Multi-run training strategy

A build request can include multiple `runs`:

- `single`: train one class group from selected datasets
- `hybrid`: train one combined multi-class model using all class groups

4. Export options

Per run, the builder can enable:

- ONNX
- TensorRT `.engine`
- TFLite

Supported precision selections in the current code include:

- FP32
- FP16
- INT8

INT8 export also carries calibration options.

5. Remote execution through SSH

The backend can:

- generate the bundle
- upload the ZIP to a remote host
- unzip and launch it
- check job status
- send limited control commands
- download log and packaged artifacts

Repository layout

```
yolo_web_builder_v4/
├─ backend/
│  ├─ app.py
│  ├─ requirements.txt
│  ├─ core/
│  │  ├─ pipeline_generator.py
│  │  ├─ security.py
│  │  ├─ spec_models.py
│  │  └─ ssh_runner.py
│  ├─ static/
│  │  ├─ app.js
│  │  └─ style.css
│  ├─ templates/
│  │  └─ index.html
│  └─ jobs/                    # saved projects, generated bundles, logs,
artifacts
├─ pipeline_templates/
│  ├─ 01_train.py
│  ├─ 02_compression.py
│  ├─ 03_export.py
│  ├─ root.py
│  ├─ requirements.txt
│  ├─ requirements_pipeline.txt
│  ├─ config/
│  ├─ scripts/
│  └─ tools/
├─ Dockerfile
├─ docker-compose.yml
├─ .env.example
├─ start.sh
└─ README.md
```

Requirements

For the web server

Recommended:

- Docker + Docker Compose

Alternative:

- Python 3.11+

For remote bundle execution

The remote Linux machine should have:

- Python executable available, for example `python3` or `/path/to/venv/bin/python`
- `unzip`
- network access for Python package installation if the environment is not pre-provisioned
- GPU / CUDA / TensorRT / TensorFlow compatibility as required by the export mode you enable

Quick start

Option A: Docker Compose (recommended)

```
cp .env.example .env
chmod +x start.sh
./start.sh
```

Or directly:

```
docker compose up -d --build
```

Open:

```
http://localhost:8000
```

Why Docker is the default path here

The repository already includes:

- `Dockerfile`
- `docker-compose.yml`
- bind-mount support for `backend/jobs`
- UID/GID mapping through `.env`

That makes project persistence more predictable on Linux hosts.

Option B: Local Python run

```
cd backend
python -m venv .venv
source .venv/bin/activate
```

```
pip install -r requirements.txt
uvicorn app:app --reload --host 0.0.0.0 --port 8000
```

Open:

```
http://localhost:8000
```

Environment variables

The supplied `.env.example` currently exposes:

- `PORT`: published service port
- `APP_UID`: host UID for bind-mounted write permissions
- `APP_GID`: host GID for bind-mounted write permissions
- `TRUST_PROXY_HEADERS`: reverse-proxy header behavior

How the workflow operates

Mode 1: Build bundle only

1. Open the web UI.
2. Fill in task name, datasets, and run targets.
3. Configure training and export options.
4. Click **Save** if you want the project to persist in the server-side project list.
5. Click **Submit / Build bundle**.
6. Download the generated ZIP from the backend.

Mode 2: SSH run

1. Configure the same project content.
2. Provide SSH connection details.
3. Submit in SSH mode.
4. The backend will:
    - build the bundle ZIP
    - upload it to the remote machine
    - unzip it into a remote work directory
    - resolve the requested Python executable
    - start the run and track status
5. Download the remote log or packaged artifacts from the provided endpoints.

Saved project behavior

Project persistence is handled server-side under:

```
backend/jobs/projects/
```

Important behavior visible in the code:

- a project appears in the home list only after **Save**
- `task` must be unique
- project index is written to `backend/jobs/projects/index.json`
- in Docker mode, `backend/jobs` is bind-mounted so saves should appear on the host immediately

Bundle outputs

The generated bundles include scripts and templates from `pipeline_templates/`.

Based on the current generator and helper scripts, the bundle is designed to produce:

- trained weights
- evaluation outputs
- exported ONNX / engine / TFLite files when selected
- packaged artifacts ZIP

The backend also exposes these download routes after SSH execution:

- `/jobs/{job_id}/artifacts.zip`
- `/jobs/{job_id}/log.txt`

API surface

The current FastAPI app exposes endpoints for:

- `/api/build_bundle`
- `/download/{job_id}.zip`
- `/api/ssh_run`
- `/api/ssh_status`
- `/api/ssh_control`
- `/api/ssh_exec`
- `/api/project_list`
- `/api/project_get/{project_id}`
- `/api/project_save`
- `/api/project_delete`
- `/jobs/{job_id}/artifacts.zip`
- `/jobs/{job_id}/log.txt`

Operational notes

Permissions

If `backend/jobs` does not persist correctly in Docker mode, check:

- `APP_UID`
- `APP_GID`
- host directory write permissions

The code explicitly fails early when `JOBS_DIR` is configured but not writable.

SSH authentication

The SSH runner supports:

- password authentication
- private key authentication
- encrypted key handling with passphrase through the password field

Engine portability

TensorRT `.engine` files should be treated as **non-portable deployment artifacts**. In practice, an engine is tied to the runtime environment in which it is built, including the target platform, TensorRT version, CUDA / driver stack, and often the GPU architecture itself. NVIDIA explicitly notes that serialized TensorRT engines are not portable across platforms, and without hardware compatibility mode they are also not portable across different GPU architectures. For reliable deployment, build the `.engine` file on the actual target system whenever possible.

TFLite portability

Unlike TensorRT `.engine` files, `.tflite` models do **not** have the same environment-bound portability limitation. A `.tflite` file is generally portable across systems as long as the target machine has a compatible TensorFlow Lite / LiteRT runtime. In the common CPU-only path, inference is executed with CPU kernels, so deployment typically only requires a compatible CPU runtime rather than device-specific engine rebuilding. This is why `.tflite` is usually the more portable export format for CPU inference workflows.

INT8 export

INT8 export depends on calibration data and on the target runtime stack. Actual success still depends on the target machine environment.

Known scope of this repository

This repository is currently focused on:

- YOLO detection workflows
- bundle generation and orchestration
- lightweight web control plane

It is **not** a full training environment by itself; it is a builder/orchestrator for a generated runtime bundle.

License